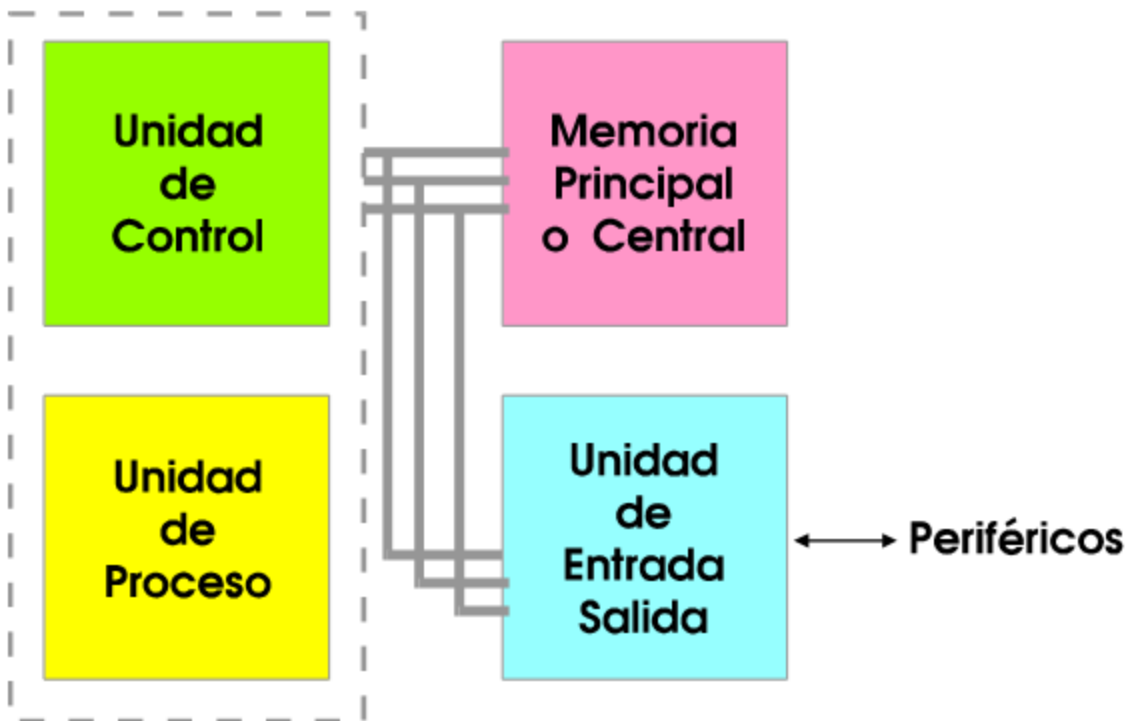


1. ● Arquitectura de computadoras

■ UNIDAD 1 —Esquema Ordenador

El **Esquema Ordenador** es el modelo funcional que describe cómo está constituida y cómo trabaja una computadora. Sirve para comprender el funcionamiento de un microprocesador, identificar los tipos de memoria, entender la importancia de los buses y relacionar estos conceptos con el equipo de computación real del usuario.

Está constituido funcionalmente por **4 bloques fundamentales**:



Desde el punto de vista físico, estos elementos son dispositivos electrónicos que se conectan entre sí para transferirse información en formato de **variable binaria**, representada por un parámetro eléctrico (la corriente) que se transmite a través de cables.

◆ El modelo Von Neumann

La organización de estos cuatro bloques constituye lo que se conoce como **modelo o arquitectura de Von Neumann**, presente (con pocas variaciones) en la mayoría de la bibliografía y en la mayoría de las computadoras actuales.

Como contraposición existe la **arquitectura Harvard**, propia de muchos microprocesadores RISC, en la que el bus de datos y el bus de instrucciones están separados, cada uno llega al

procesador por caminos diferentes y por lo tanto se procesan en paralelo (a diferencia de Von Neumann, donde datos e instrucciones comparten el mismo bus/memoria).

Actividad de repaso relacionada: identificar los buses en la propia PC; explicar qué son la arquitectura Harvard y Von Neumann.

◆ Unidad de Control

Es el elemento encargado de:

- Leer las instrucciones desde la memoria central.
- Decodificarlas o interpretarlas.
- Ordenar a la Unidad de Proceso qué operación debe realizarse.

Se la podría considerar el "**cerebro**" del **Esquema Ordenador**. Para cumplir su función utiliza un par de registros:

- **Contador Ordinal** (o **Puntero de Instrucciones**): siempre "apunta" a la próxima instrucción a ejecutar. La búsqueda de la instrucción en la memoria, en la posición indicada por este registro, se conoce como **ciclo de fetch** (búsqueda).
- **Registro de Operación**: almacena el contenido de la instrucción leída para poder decodificarla ("saber" qué hace) y ejecutarla.

Una vez decodificada la instrucción, la Unidad de Control, mediante un **secuenciador** (generador de secuencias), establece las conexiones lógicas necesarias para el desarrollo y ejecución de la instrucción. Por ejemplo, en una suma, la Unidad de Control hace que la UAL tome dos datos almacenados temporalmente en dos registros, y el resultado se almacena a su vez en otro registro.

Registro: elemento de almacenamiento de información binaria de uno o más bits. Según su contenido se clasifican en registros generales, de operación, de segmento, punteros, índices, etc. El tamaño de los registros de la CPU suele coincidir con el tamaño de la palabra de memoria, para que cualquier dato pueda pasar de un registro a la memoria y viceversa.

◆ Unidad de Proceso

Unidad de Proceso o Unidad Aritmético Lógica (UAL)

Es la encargada de **ejecutar** las instrucciones interpretadas por la Unidad de Control. Para ello debe reconocer la operación que presenta la instrucción y los operandos con los que debe realizarse.

- Internamente es un conjunto de **compuertas lógicas** (suma, producto y complementación — álgebra de Boole).
- Realiza operaciones **aritméticas** (sumas, restas, multiplicaciones, etc.) y **lógicas** (comparaciones, NOT, AND, OR, etc.).
- Se comunica con las otras unidades a través del **bus de datos**.
- Trabaja con registros internos que contienen los operandos y, finalmente, el resultado.
- El resultado puede enviarse a un registro de memoria o a un registro de una Unidad de Entrada/Salida, llamado **puerto**.

Puerto: registro ubicado en una Unidad de Entrada/Salida. Al igual que los registros de memoria, se identifica mediante una dirección.

Representación alternativa de la UAL: recibe datos por un **Registro de Entrada**, los procesa en el **Circuito Operacional**, mantiene un **Registro de Estado** y entrega el resultado a través de un **Acumulador**.

El "programa": el Esquema Ordenador no tiene razón de ser sin un programa que lo controle y que esté residente en la Memoria Principal. Un programa es un algoritmo constituido por instrucciones que se ejecutan de manera secuencial (o según el orden que determine el propio algoritmo). Por eso, la máquina realiza siempre la misma tarea:

Lee – Interpreta – Ejecuta

- **¿De dónde lee?** De la Memoria Principal, donde reside el programa en ejecución.
- **¿Qué lee?** Una instrucción por vez (proceso indivisible).
- **¿Qué interpreta?** La instrucción leída.
- **¿Qué ejecuta?** La instrucción interpretada.

Esta actividad la realiza siempre y permanentemente, y es llevada a cabo por el microprocesador, que constituye la Unidad de Control y de Proceso.

Lenguaje máquina y lenguaje ensamblador

- El procesador ejecuta instrucciones definidas en **lenguaje máquina** (ceros y unos, de forma específica e inequívoca).
- El **lenguaje ensamblador** es una representación de esas instrucciones usando letras y números en lugar de lógica binaria, para facilitar la escritura de programas. Un programa (ensamblador) traduce este lenguaje a lenguaje máquina.
- Un microprocesador reconoce un **número limitado de instrucciones**; si lee una que no puede interpretar, el equipo puede "colgarse".
- Cada instrucción se ejecuta en un **número fijo de pulsos o períodos de reloj**.
- Las instrucciones no necesariamente tienen el mismo tamaño (en bits o bytes).

- Causas típicas de "cuelgue": memoria que no funciona bien, programa mal grabado en disco, disco defectuoso, problemas de velocidad entre microprocesador y memoria.

Ejemplo de instrucciones en ensamblador: `MOV AX,CX / MOV DS,DX / NOP / CLI / ADD DL,1 / INC SI / CMP DI,AX / MOV DL,[DI] / INT 10h .`

Set de Instrucciones: información que cada fabricante (Intel, Motorola, VIA, IBM, AMD, etc.) suministra sobre su microprocesador: qué instrucciones reconoce, con qué operadores las ejecuta y en cuántos ciclos de reloj.

A mayor frecuencia de reloj, menor período, por lo tanto las instrucciones (que se ejecutan en un número fijo de pulsos) se ejecutan más rápido → mayor frecuencia = mayor velocidad. Sin embargo, la frecuencia no es el único parámetro que determina las prestaciones, y el aumento de frecuencia tiene un límite físico (de allí surge el concepto de **overclocking**, con sus riesgos).

Microprocesadores: clasificación y parámetros

Los microprocesadores se dividen en dos grandes familias:

- **CISC** (Complex Instruction Set Computer – set complejo de instrucciones).
- **RISC** (Reduced Instruction Set Computer – set reducido de instrucciones).

Características típicas de un RISC:

- Conjunto reducido de instrucciones, generalmente del mismo tamaño (1 palabra de memoria).
- Conjunto reducido de modos de direccionamiento (a veces solo direccionamiento directo).
- 1 instrucción, 1 ciclo (mayoría optimizadas para un solo ciclo; no suelen usar microcódigo, están cableados).
- Muchos registros, organizados en bancos (por ejemplo, de 16 en 16), de propósito general (register file).
- Trabajo intensivo con registros.
- **Pipeline:** varias instrucciones pueden terminar simultáneamente en cada ciclo (procesadores superescalares).
- Arquitectura tipo **Harvard** (bus de datos y de instrucciones separados).
- Requieren compiladores complejos para optimizar el uso de los pipelines.

Familias de microprocesadores: los miembros más modernos entienden el código máquina de los más antiguos (**compatibilidad binaria**), aunque no siempre son compatibles en pines.

Parámetros a considerar en un microprocesador:

- Tecnología del set: RISC o CISC.
- Familia a la que pertenece (o de qué procesador es clon).
- Velocidad en MHz o GHz (la velocidad interna a veces no coincide con la externa).
- Palabra interna y externa (tamaño de registros vs. ancho del camino de datos con la memoria).
- Coprocesador matemático (FPU) integrado o no.
- Caché interna o no.
- Consumo y voltaje de trabajo (importante en portátiles; modo *sleep*).
- Complejidad de instrucciones, uso de pipeline, cantidad de unidades de proceso internas, compatibilidad, acceso a distintos tipos de buses.

Familia de PC compatibles: genéricamente denominada **80x86** (8088, 8086, 80186, 80286, 80386 SX/DX, 80486, Pentium I-IV, Pentium Pro, Pentium MMX, Klamath, Deschutes, P7/Merced, etc.), con competidores como AMD, Cyrix, NexGen, VIA.

Otras familias: Motorola 680x0 (Mac, Atari ST, Amiga clásicos), PowerMac (RISC).

Multiprocesadores de escritorio:

- **Funcional:** procesadores distintos en capacidad y función (el más habitual).
- **Simétrico (SMP):** varios procesadores iguales; se clasifican por grano (fino o grueso) según el nivel de división de tareas.
- Según la memoria: **memoria compartida** (SMP, no escalable, problemas de coherencia), **memoria distribuida** (MPP, proceso masivamente paralelo) y **proceso paralelo escalable** (híbrido de ambas).

◆ **Memoria Central**

También llamada **Memoria Principal**, es la unidad donde se almacenan los **datos e instrucciones** necesarios para realizar un proceso. Es rápida y está estrechamente ligada a las unidades funcionales más rápidas de la computadora. Aquí deben cargarse los programas para poder ejecutarse.

- Formada por circuitos electrónicos capaces de almacenar solo dos valores (0 o 1) por celda.
- **Palabra de memoria:** menor conjunto de celdas que se pueden leer o escribir de una vez (no se accede a una sola celda, sino a un conjunto consecutivo).
- **Dirección de memoria:** número que identifica de forma única cada posición (palabra) de memoria; permite el acceso directo a cualquier posición → por eso se dice que es una **memoria de acceso directo** o accesible por dirección.

- Todos los programas se ejecutan desde la Memoria Principal, aunque residan originalmente en memorias auxiliares (discos, CD, pen drive, etc.), porque deben cargarse primero en memoria para ejecutarse.
- Diferencia de velocidad: acceso a disco $\approx$ milisegundos (10^{-3} s) vs. acceso a memoria $\approx$ nanosegundos (10^{-9} s) — una diferencia de 6 órdenes de magnitud.

Ejemplo: una memoria de 16x8 tiene 16 registros de 8 bits (1 byte) cada uno; para direccionar 16 posiciones se necesitan 4 bits de dirección ($2^4=16$) y por lo tanto 4 líneas en el bus de direcciones.

El **bus de direcciones** selecciona tanto registros de memoria como puertos de los dispositivos de E/S (los puertos también deben direccionarse; ejemplo: direcciones de puertos serie/paralelo en hexadecimal: 3F8, 2F8, 2E8, 378, 278).

Tipos de memoria principal: RAM y ROM

- **RAM** (Random Access Memory): puede leerse y escribirse; es **volátil** (pierde su contenido al desenergizarse).
- **ROM** (Read Only Memory): solo lectura. En la PC constituye la **ROM BIOS**, que contiene el programa que se lee al arrancar (ya que la RAM pierde su contenido).

Subtipos de RAM:

- **SRAM** (Static RAM): más rápida, más cara y más voluminosa. Se usa en las memorias caché.
- **DRAM** (Dynamic RAM): más lenta y económica. Constituye la Memoria Principal.

Velocidad de la memoria

Tres parámetros clave:

- **Tiempo de acceso (t_A):** tiempo máximo para leer o escribir una posición.
- **Tiempo de ciclo (t_C):** tiempo mínimo entre dos lecturas consecutivas.
- **Ancho de Banda (AB):** cantidad de palabras transferidas entre memoria y CPU por unidad de tiempo ($AB = 1/t_C$).

Jerarquía de memoria y memoria virtual

Si el programa es más grande que la Memoria Principal, se usa la técnica de **Memoria Virtual**: se guarda el programa y sus datos en memoria auxiliar, manteniendo en Memoria Principal solo la parte implicada en la ejecución en ese momento.

Como la Unidad de Control opera unas 10 veces más rápido que la Memoria Principal, se introduce **memoria caché** entre la CPU y la memoria principal para no desperdiciar tiempo.

Jerarquía (de más rápida/cara/pequeña a más lenta/barata/grande):

1. Registros CPU
2. Caché (SRAM)
3. Memoria Principal (DRAM)
4. Memoria Masiva (discos, etc.)

Parámetros de caracterización en cada nivel: **capacidad de almacenamiento (s)**, **tiempo de acceso (t)**, **ancho de banda (b)** y **costo (c)**. En general: a mayor velocidad, mayor ancho de banda, mayor costo y menor capacidad.

Funcionamiento físico de la memoria (DRAM)

- Cada bit se almacena en un pequeño condensador; los bits se organizan en filas y columnas.
- Para leer un bit se debe indicar dirección de fila y de columna.
- Tras el acceso, el chip necesita un **tiempo de precarga** antes de poder ser accedido nuevamente.
- La DRAM debe **refrescarse** periódicamente para mantener la carga (se puede refrescar fila por fila).
- Un tiempo de acceso $t=60$ ns requiere casi el doble de tiempo para completar un ciclo completo de lectura/escritura.

Técnicas para acelerar el acceso:

- **FPM-RAM** (Fast Page Mode): accesos rápidos dentro de una misma página, una vez indicada la fila.
- **EDO RAM** (Extended Data Out): almacena la salida en un registro el tiempo suficiente para que la CPU la lea.
- **Modo burst**: la CPU toma grupos de direcciones (por ejemplo, de 4 en 4) en vez de una por vez.
- **Modo entrelazado**: la memoria se divide en bancos; mientras se accede a uno, otro puede precargarse o refrescarse.

Cachés: tipos y organización

Se basan en el **principio de localidad** (espacial y temporal).

Según cómo actualizan la memoria principal:

- **Write-through**: escribe en memoria principal cada vez que se escribe en la caché (asegura coherencia total, pero usa más el bus).

- **Write-back:** escribe en memoria principal solo cuando una posición debe salir de la caché.
- **Burst-pipeline:** funciona de modo síncrono, ligeramente más rápida que las otras.

Según organización (mapeo de bloques a líneas):

- **Mapeo directo:** la dirección se divide en offset, línea y tag; problema de *data thrashing* (dos bloques compiten por la misma línea).
- **Plenamente asociativa:** cualquier bloque puede ir a cualquier línea; requiere comparar todos los tags (se usa el **TLB**, translation lookaside buffer).
- **Asociativa por conjuntos:** la caché se divide en conjuntos con varias líneas cada uno; combina las dos anteriores.

Puede haber **caché de datos** y **caché de instrucciones** separadas.

Actividades relacionadas del capítulo: averiguar la velocidad de acceso a disco rígido y a memoria principal; si una máquina puede funcionar sin ROM; identificar chipset y clock de la memoria propia.

◆ Unidades de Entrada/Salida

Engloban todas las **interfaces con los dispositivos periféricos**: puertos serie y paralelo, placa de video, puertos USB, conexión al teclado, puerto de juegos, conectores IDE (para disqueteras, discos y lectoras de CD), etc.

Función: comunicar el Esquema Ordenador con el exterior a través de los periféricos correspondientes. Para acceder a ellos se identifican mediante la **dirección de sus puertos**.

Clasificación de periféricos

Memorias Auxiliares:

- Discos **Magnéticos**: flexibles, rígidos.
- Discos **Ópticos**: CD-ROM, CD R/W, DVD R/W.
- **Electrónicos o de Estado Sólido**: pen drives, MP3, MP4.
- Cintas.

Dispositivos de Comunicación:

- Teclado, Monitor, Mouse, Impresora, Plotter, Scanner, Lápiz óptico, Touch Screen, etc.

Importante: el hecho de que un dispositivo sea periférico no implica que esté físicamente fuera del gabinete.

♦ Buses del sistema. Importancia de los Buses

¿Qué son los buses?

Los **buses** son el "conjunto de cables" que conecta y permite transferir información binaria entre los elementos del Esquema Ordenador. **NO tienen capacidad de almacenar información, solo la transmiten** (son cables). Se llaman "buses" porque son comunes a todos los elementos que conforman el Esquema Ordenador. Físicamente se implementan como "pistas" de cobre sobre el material epoxi de la placa (motherboard).

Los tres buses del Esquema Ordenador

1. Bus de datos

- Transporta los datos que se transfieren entre unidades.
- El número de líneas (bits) determina el **tamaño de palabra** del ordenador, es decir, el tamaño del dato que se puede transmitir.
- Es **bidireccional**: los mismos cables transmiten información hacia dentro o hacia fuera en instantes diferentes.

2. Bus de direcciones

- Transporta la dirección de la posición de memoria o del periférico involucrado (de dónde procede o hacia dónde se dirige el dato).
- Permite la comunicación entre el procesador y las celdas de memoria RAM: el procesador envía la dirección a leer, y recibe el contenido por el bus de datos.
- El **tamaño de este bus define la cantidad de memoria RAM** que la CPU puede gestionar. Ejemplos:
 - 10 bits $\rightarrow 2^{10} = 1024$ palabras (1 KB)
 - 16 bits $\rightarrow 2^{16} = 65536$ palabras (65 KB)
 - 32 bits $\rightarrow 2^{32} = 4.294.967.296$ palabras (4 GB)
- Es **unidireccional** (selecciona origen o destino).

3. Bus de control

- Transporta las señales de control y de estado: indica dirección de la transferencia, controla la temporización de eventos, transmite señales de interrupción, etc.
- Sincroniza y controla todos los componentes.
- Las señales de **control** parten de la Unidad de Control hacia el resto; las señales de **estado** parten del resto de los elementos hacia la Unidad de Control.
- Es de naturaleza **heterogénea**, depende del microprocesador.

Bus interno (de CPU) vs. bus de expansión

- **Bus interno o de CPU:** conecta la CPU con la memoria; destaca por su velocidad.
- **Bus de expansión:** conecta la CPU con el resto de los elementos (es una prolongación del bus interno). Consiste en conectores donde se enchufan tarjetas/placas específicas para cada periférico (video, controladora de disco, etc.). Forma parte del bus del sistema (control + direcciones + datos), conectado mediante el **interfaz de periféricos**.
- **Bus local:** va conectado directamente a las patillas de datos del microprocesador; los periféricos en bus local tienen la misma velocidad de acceso que la memoria.

Parámetros que definen un bus

- **Ancho del camino de datos:** de 8 a 64 bits (o incluso 1 bit en bus serie); cuántos bits se intercambian simultáneamente.
- **Velocidad:** en MHz (veces que cambia de estado por segundo) o en Mbps (velocidad real de transferencia).
- **Bus mastering:** una tarjeta "bus master" (inteligente, con microprocesador propio) puede sustituir a la CPU como controladora, accediendo a memoria y periféricos sin detener la CPU.
- **Familia:** compatibilidad ascendente entre buses "hermanos".
- **Data streaming o modo burst:** transmisión rápida de datos secuenciales, con menos "reconocimientos" (Ack's).
- **Plug'n'Play:** configuración automática del periférico sin ajustes manuales de interrupciones/direcciones.
- **Hot swapping:** agregar/quitar periféricos con el equipo encendido, con reconfiguración automática.

Ejemplos de buses en PC

- **ISA** (Industry Standard Architecture): 8/16 bits, sin bus mastering, 8 MHz; usado hoy solo para periféricos lentos.
- **EISA y MicroChannel:** 32 bits, admiten bus masters, hasta 33 MBps, configuración automática.
- **Local Bus / VLB (VESA Local Bus):** líneas conectadas directamente al procesador a su misma velocidad; hasta 3 bus masters, hasta 250 MBps.
- **PCI:** bus local de Intel, 66-100 MHz, funciona a 3.3V o 5V, modo burst, multiplexado (47 pines), hasta 10 periféricos, configuración automática, también versión de 64 bits.
- **AGP** (Accelerated/Advanced Graphics Port): vía dedicada entre memoria principal y tarjeta gráfica 3D, bus de 32 bits a 66 MHz (el doble que PCI); versiones más rápidas (133 MHz, 533 MBps) llegan a 1 GB/s.
- **PCMCIA / PCCard:** para tarjetas de portátiles (memorias flash, discos), tipos I/II/III, versión CardBus con bus-mastering de 32 bits.

- **SCSI** (Small Computer System Interface): periféricos "inteligentes" con interfaz incluido, conexión en cadena; versiones SCSI-2, UltraWide, SCSI3.
- **NuBus** (Mac): 16/32 bits, admite bus master.
- **PDS** (Processor Direct Slot): equivalente Mac del bus local.
- **Otros**: s-bus y M-bus (Sun), VMEbus (Silicon Graphics), TurboChannel (DECstation).
- **Buses serie**: Access.bus (125 KBps), FireWire (hasta 20 MBps), **USB** (Universal Serial Bus).
- **Futuro**: conexiones de fibra óptica (Fibre Channel), del orden del GBps.

Importancia de los buses

Los buses son el **canal común de comunicación** entre todos los bloques del Esquema Ordenador (Unidad de Control, Unidad de Proceso, Memoria y Unidades de E/S). Sin ellos no habría transferencia de datos, direcciones ni señales de control/sincronización entre los componentes. El tamaño y la velocidad de los buses son determinantes de las prestaciones globales del sistema: el bus de datos define el tamaño de palabra manejable, el bus de direcciones define la cantidad máxima de memoria direccionable, y el bus de control asegura que todos los componentes trabajen sincronizados.

♦ Unidades de Ejecución y de Interfaz Bus

Corresponde a la arquitectura interna del microprocesador (ejemplo desarrollado en el Anexo sobre el **8086/8088**), que se divide en dos grandes bloques:

Unidad de Ejecución (EU – Execution Unit)

Contiene:

- **Registros de uso general** (AX, BX, CX, DX, y sus mitades AH/AL, BH/BL, CH/CL, DH/DL), además de SP, BP, SI, DI.
- **Registros temporales.**
- La **Unidad Aritmético-Lógica (UAL)**, que realiza las operaciones.
- **Indicadores** (flags), que reflejan el resultado/estado de las operaciones.
- Un **Sistema de control de la unidad de ejecución**, que coordina el trabajo interno.
- Se comunica internamente mediante el **bus interno de 16 bits**.

Es la parte del microprocesador que efectivamente **ejecuta las instrucciones** (opera con los datos).

Unidad de Interfaz con el Bus (BIU – Bus Interface Unit)

Contiene:

- Los registros de segmento y comunicación interna: **CS, DS, ES, SS, IP** (registros de comunicación interna).
- Un **sumador** que combina Segmento + Offset para calcular direcciones físicas.
- La **Lógica de Control del Bus**, que gestiona los **buses externos**.
- La **Cola de instrucciones** (instruction queue): 4 bytes en el 8088, 6 bytes en el 8086 — permite ir buscando instrucciones por adelantado mientras la EU ejecuta la actual.
- Genera y gestiona el **bus de direcciones** (20 bits en el 8086/8088) y el **bus de datos** (8 bits en el 8088, 16 bits en el 8086) hacia el exterior del chip.

Es la parte del microprocesador encargada de **comunicarse con el exterior** (memoria y periféricos) a través de los buses externos, mientras la Unidad de Ejecución trabaja internamente.

Esta división (EU / BIU) permite que, mientras la EU ejecuta una instrucción, la BIU pueda ir a buscar las siguientes por adelantado (mediante la cola de instrucciones), optimizando el rendimiento — un antecedente de las técnicas de **pipeline** usadas en procesadores más modernos.

◆ Esquema de Operación Hardware y Software. Capas de trabajo

Nivel físico / hardware

- El Esquema Ordenador está compuesto por dispositivos electrónicos (Unidad de Control, Unidad de Proceso, Memoria, Unidades de E/S) interconectados por los buses.
- El conjunto de **Unidad de Control + Unidad de Proceso** constituye el **microprocesador**.
- La **placa madre (motherboard)** es, en términos de implementación, "el ordenador": es el componente fundamental y la unidad mínima intercambiable para modificar las características básicas del equipo (principio de "inviolabilidad de la placa madre": no se conecta nada que la placa no permita).
 - En la placa madre se encuentran: el microprocesador (o su zócalo), el coprocesador matemático (hoy integrado), la memoria RAM y ROM, los slots de expansión, el **chipset** completo, y a veces chips de interfaz para teclado, red, sonido o video.
 - La tendencia es hacia mayor **integración**: menos chips, menor consumo, mayor velocidad.
 - Formatos históricos: baby AT, LPX, ATX.
- El **Chipset** incluye la lógica principal del sistema, controladores de interrupciones, controladores DMA, el controlador de caché y los buffers de bus; puede incluir también el

oscilador (controla la operación síncrona de la placa). La tendencia es reducirlo a uno o dos chips (ej. chipset Triton FX de Intel).

- El **Reloj**: sincroniza el trabajo de todo el Esquema Ordenador; su frecuencia se mide en Hertz (ciclos/segundo). A mayor frecuencia, menor período, y por lo tanto mayor velocidad de ejecución de instrucciones (dentro de los límites físicos del dispositivo). Está asociado directamente al microprocesador.
 - Factores de multiplicación: 1 KHz = 1.000 Hz; 1 MHz = 1.000.000 Hz; 1 GHz = 1.000.000.000 Hz.
 - Relación Frecuencia = 1/Período (a mayor frecuencia, período más corto: ms, μ s, ns respectivamente).
 - Evolución histórica: Intel 8080 (Altair, 2 MHz) → IBM PC original (4,77 MHz) → Pentium (100 MHz, 1995) → Pentium 4 (3 GHz, 2002) → arquitecturas multinúcleo (2, 4, 6, 8 núcleos), ya que la frecuencia por núcleo dejó de aumentar significativamente.

Nivel lógico / capas de trabajo (de hardware a software)

El esquema de operación puede pensarse en **capas**, desde lo más cercano al hardware hasta lo más cercano al usuario:

1. **Nivel físico/eléctrico**: señales binarias (corriente) transmitidas por los buses entre los bloques del hardware.
2. **Nivel de lenguaje máquina**: las instrucciones que el microprocesador reconoce, codificadas en ceros y unos según su **Set de Instrucciones**.
3. **Nivel de lenguaje ensamblador**: representación simbólica (letras y números) del lenguaje máquina, traducida por un ensamblador.
4. **Nivel de programa/software**: conjunto de instrucciones (algoritmo) que reside en la Memoria Principal y que el Esquema Ordenador procesa siempre mediante el ciclo **Lee – Interpreta – Ejecuta**.
5. **Nivel de sistema operativo / BIOS**: dado que la RAM es volátil, es la **ROM BIOS** la que contiene el programa que se ejecuta al arrancar el equipo, permitiendo luego cargar el sistema operativo y los programas de usuario en la Memoria Principal.

Interacción hardware-software

- Ningún programa puede ejecutarse si no está cargado en la Memoria Principal.
- El hardware (microprocesador, buses, memoria, E/S) provee la infraestructura física; el software (BIOS, sistema operativo, programas de aplicación) define **qué** se ejecuta y **en qué orden**.
- La velocidad final percibida por el usuario depende de la combinación de todos los niveles: frecuencia de reloj, tipos de memoria y caché, buses disponibles, arquitectura del

microprocesador (CISC/RISC, pipeline, superescalar) y eficiencia del software (por ejemplo, compiladores que optimizan el código para aprovechar los pipelines).

UNIDAD 2 — Microprocesadores 8086/8088 y Memorias RAM

♦ 2.1 — Historia del 8086/8088

Intel lanzó el 8086 en junio de 1978: el primer microprocesador de 16 bits. En 1979 apareció el 8088 (igual internamente pero con bus de datos externo de 8 bits). En 1980 se sumaron los coprocesadores 8087 (matemático) y 8089 (E/S).

El hito más importante fue la elección del 8088 por IBM en 1981 para su primera PC, consolidando la arquitectura x86 como estándar de la industria.

♦ 2.2 — Arquitectura interna: EU y BIU

Cada procesador se divide en dos sub-unidades:

- **EU (Execution Unit):** ejecuta todas las operaciones. Idéntica en 8086 y 8088.
- **BIU (Bus Interface Unit):** accede a datos e instrucciones del exterior. El 8088 tiene bus de datos externo de 8 bits; el 8086, de 16 bits.

Esta división permitió que Intel solo tuviera que rediseñar la BIU para producir el 8088, ahorrando esfuerzo de desarrollo (la EU se mantuvo intacta). Además, gracias a que el 8088 comparte con el 8085 la forma de multiplexar datos y direcciones, además de requerimientos de control y timing similares, pudo reemplazarlo en sistemas ya existentes sin necesidad de rediseñar todo el hardware circundante.

♦ 2.3 — Registros del microprocesador (1 al 14)

Los registros son pequeñas unidades de memoria ubicadas dentro del propio microprocesador, no en la RAM. Son la forma más rápida de almacenamiento que existe (más rápidos incluso que la caché), porque el micro los usa como su "espacio de trabajo" inmediato: ahí guarda los operandos antes de operarlos, los resultados de las operaciones, y las direcciones que necesita para acceder a memoria.

El 8086/8088 tiene, en total, **14 registros**: 8 registros de uso general (de 16 bits, viven en la EU), 4 registros de segmento y el puntero de instrucción IP (viven en la BIU, se usan para calcular direcciones físicas de memoria), y el registro de banderas (Flags). Cuatro de los registros generales (AX, BX, CX, DX) pueden subdividirse en dos registros de 8 bits (por ejemplo AX en AH y AL), lo que permite operar con bytes individuales cuando no se necesita la

palabra completa. Los otros cuatro (SP, BP, SI, DI) no se pueden subdividir y se usan principalmente para direccionamiento (apuntar a posiciones de memoria, la pila, o recorrer cadenas de datos).

Registros de uso general:

1. **AX (AH/AL) — Acumulador:** Aritmética, IN/OUT, multiplicaciones/divisiones. Genera instrucciones de 1 byte menos que otros registros. AL equivale al acumulador del 8080/8085.
2. **BX (BH/BL) — Base:** Direccionamiento indirecto. BX/SI/DI usan DS; BP usa SS.
3. **CX (CH/CL) — Contador:** Bucles (LOOP), cadenas con prefijo REP, desplazamientos/rotaciones (usa CL).
4. **DX (DH/DL) — Datos:** Multiplicaciones/divisiones junto con AX, puertos E/S en modo indirecto.
5. **SP — Puntero de pila:** Solo puntero de pila. PUSH resta 2; POP suma 2. No subdivisible.
6. **BP — Puntero base:** Direccionamiento indirecto dentro de la pila. No subdivisible.
7. **SI — Índice fuente:** Puntero fuente en operaciones con cadenas. También direccionamiento indirecto.
8. **DI — Índice destino:** Puntero destino en operaciones con cadenas. También direccionamiento indirecto.

Registros de segmento — viven en la BIU:

Para resolver la limitación de sus registros internos de 16 bits (que solo pueden direccionar hasta 64 KB por sí solos) y poder acceder a su bus de direcciones de 20 bits (hasta 1 MB de RAM), el procesador 8086/8088 utiliza la segmentación. La memoria se divide dinámicamente en bloques de hasta 64 KB llamados segmentos.

9. **CS (Code Segment — Segmento de Código):** Apunta a la dirección base del segmento donde están almacenadas las instrucciones (el código) del programa que se está ejecutando. Compañero: trabaja en conjunto con el puntero de instrucción IP para buscar la siguiente instrucción a ejecutar (CS:IP).
10. **DS (Data Segment — Segmento de Datos):** Apunta al segmento donde se guardan las variables y datos generales del programa. Compañero: es el segmento por defecto cuando se usan registros como BX, SI o DI para acceder a memoria.
11. **SS (Stack Segment — Segmento de Pila):** Apunta al segmento reservado para la pila (stack), la estructura de memoria de tipo LIFO usada para guardar direcciones de retorno de funciones, parámetros y guardar el contexto. Compañero: trabaja junto con los punteros de pila SP y BP (SS:SP o SS:BP).
12. **ES (Extra Segment — Segmento Extra):** Funciona como un segmento adicional de datos. Se utiliza como destino en operaciones con cadenas de bloques de datos o cuando un

programa necesita manejar un segundo bloque de datos grande sin alterar DS.
Compañero: en operaciones con cadenas, trabaja junto con el registro índice destino DI (ES:DI).

Puntero de instrucción — vive en la BIU:

13. **IP (Instruction Pointer)**: Apunta a la próxima instrucción a ejecutar dentro del segmento de código. Trabaja siempre junto a CS (CS:IP). Es el "Contador Ordinal" mencionado en la Unidad de Control (sección 1.3).

Registro de banderas:

14. **FLAGS (Registro de Indicadores)**: Registro de 16 bits con 9 flags activos que refleja el resultado de las operaciones de la UAL (suma, resta, comparaciones, AND, OR, XOR, TEST, etc.) y controla el comportamiento del procesador. Los bits 5 y 3 siempre valen 0; el bit 1 siempre vale 1.

Flag	Bit	Significado
CF	0	Carry: acarreo (suma) o préstamo (resta) desde el bit más significativo.
PF	2	Paridad: resultado con cantidad par de bits en 1.
AF	4	Carry auxiliar: acarreo entre nibbles (ajuste decimal).
ZF	6	Zero: el resultado es cero.
SF	7	Signo: bit más significativo del resultado (0 = positivo, 1 = negativo).
TF	8	Trap: modo paso a paso (genera interrupción interna después de cada instrucción).
IF	9	Interrupción: habilita (1) o deshabilita (0) interrupciones enmascarables.
DF	10	Dirección: 0 = auto-incremento en cadenas; 1 = auto-decremento.
OF	11	Overflow: desborde en operación aritmética con signo.

◆ 2.4 — Segmentación y uso de la memoria

El 8086 tiene bus de direcciones de 20 bits → direcciona 1 MB (00000H a FFFFFH). Para reducir el campo de direcciones en las instrucciones a 16 bits, la memoria se divide en 4 segmentos de hasta 64 KB cada uno.

Dirección física = (Registro de segmento × 10H) + Desplazamiento (offset) El segmento se desplaza 4 bits a la izquierda (se agregan 4 ceros en hex) y se le suma el offset.

Tipo de acceso	Segmento por defecto	Desplazamiento
Búsqueda de instrucción	CS	IP
Operación sobre la pila	SS	SP
Acceso a dato (general)	DS	Dirección efectiva
Dato con BP como base	SS	Dirección efectiva
Cadenas — dato destino	ES	DI

- **Segmentos:** los 4 (CS, DS, SS, ES) pueden quedar separados, solapados o incluso apuntar al mismo lugar, porque el valor de cada registro de segmento lo elegís libremente — el hardware no obliga a que no se toquen.
- **Relocalización:** un programa se puede cargar en cualquier dirección de memoria sin reescribirlo, siempre que todas sus referencias sean offsets relativos a los registros de segmento (no direcciones fijas ni saltos entre segmentos hardcodeados). Así el SO lo reubica solo ajustando esos registros al cargarlo.
- **Direcciones reservadas:** 00000H–003FFH guarda la tabla de vectores de interrupción (256 entradas). FFFF0H–FFFFFFH es adonde salta el micro automáticamente al hacer RESET, normalmente con un JMP hacia la ROM BIOS para arrancar.
- **Pila:** trabaja con palabras de 16 bits (no bytes sueltos) y crece hacia direcciones decrecientes. SP siempre apunta a la última palabra metida (el tope).
 - PUSH: primero resta 2 a SP, después guarda el byte alto en SP+1 y el byte bajo en SP.
 - POP: lee el byte bajo desde SP y el alto desde SP+1, y recién ahí suma 2 a SP.

♦ ♦ 2.5 — Cola de instrucciones (prefetch)

La BIU carga instrucciones anticipadamente cuando el bus está libre, permitiendo que la EU ejecute mientras la BIU busca la siguiente.

- 8086: cola de 6 bytes, carga de a 2 bytes (bus de 16 bits).
- 8088: cola de 4 bytes, carga de a 1 byte (bus de 8 bits).

Los saltos (JMP, CALL) vacían la cola porque las instrucciones que siguen al salto no se van a ejecutar; la cola debe recargarse desde la dirección de destino. Por eso los saltos son las instrucciones más lentas (junto con multiplicaciones y divisiones).

♦ 2.6 — UAL e Indicadores (Flags)

La UAL realiza operaciones de 8 o 16 bits: suma, suma con acarreo, resta, resta con préstamo, comparaciones, AND, OR, XOR, TEST.

(Ver el detalle del registro de Flags en el punto 14 de la sección 2.3 — Registros del microprocesador).

♦ 2.7 — Modos de direccionamiento de datos

Modo	Descripción	Ejemplo
Por registro	Entre registros del mismo tamaño. El más rápido. No accede a memoria. No se permite MOV entre registros de segmento.	<code>MOV AX, BX → AX ← BX</code>
Inmediato	El dato está en la instrucción. Muy rápido (usa la cola).	<code>MOV AX, 3456H → AX ← 3456H</code>
Directo	El desplazamiento se suma al DS. 3 bytes (solo AL/AX) o 4 bytes (cualquier registro).	<code>MOV AX, 1234H → AX ← [DS×10H + 1234H]</code>
Indirecto por registro	La dirección está en BX/SI/DI (usa DS) o BP (usa SS).	<code>MOV AX, [BX] → AX ← [DS×10H + BX]</code>
Base más índice	Registro base (BX o BP) + índice (SI o DI). Para tablas.	<code>MOV AX, [BX+SI] → AX ← [DS×10H + BX + SI]</code>
Relativo por registro	Agrega desplazamiento constante al registro base/índice.	<code>MOV [BX+4], AX → [DS×10H + BX + 4] ← AX</code>
Relativo base + índice	Base + índice + desplazamiento. Para arreglos bidimensionales.	<code>MOV ARRAY[BX+SI], AX</code>
Implícito	El operando está implícito en la instrucción.	<code>HLT, LOCK, WAIT, DAA</code>

Idea clave que conecta todo: todos los modos que acceden a memoria (directo, indirecto, base, indexado, etc.) terminan resolviéndose con la fórmula ya vista:

Dirección física = (Registro de segmento × 10H) + Desplazamiento

Donde el "desplazamiento" (offset) es justamente lo que cambia según el modo: puede ser un número fijo, el contenido de un registro, o la suma de varios.

♦ 2.8 — Modos de direccionamiento de programa (saltos)

- **Directo:** la dirección destino está en la instrucción. Saltos largos entre segmentos. `JMP 10000H → CS ← 10000H, IP ← 0000H`
- **Relativo:** modifica el IP según un desplazamiento. Saltos cortos intrasegmento. `JMP 2 → IP ← IP + 2`
- **Indirecto:** la dirección está en un registro. `JMP [BX] → IP ← BX`
- **Pila (CALL/RET):** CALL guarda la dirección de retorno en la pila; RET la recupera.

♦ 2.9 — Modo Mínimo y Modo Máximo

El 8086/8088 puede configurarse en dos modos, según el pin MN/MX#:

- **Modo Mínimo:** pensado para sistemas chicos de un solo procesador. El propio micro genera directamente las señales de control del bus, sin necesitar chips adicionales de control.
- **Modo Máximo:** pensado para sistemas más complejos (multiprocesador o que requieren más señales de control). Acá el micro delega la generación de señales de control a un chip externo, y se necesitan al menos estos circuitos extra:
 - **8288 Bus Controller:** genera las señales de control del bus (lectura/escritura de memoria y E/S) a partir de los códigos de estado que entrega el 8086.
 - **8284A Clock Generator:** genera la señal de reloj del sistema a partir de un cristal externo.
 - **74HC373 (x3):** latches que demultiplexan las líneas de direcciones (recordá que direcciones y datos comparten pines en el 8086, por eso hace falta esta lógica extra).
 - **74HC245 (x2):** buffers bidireccionales para el bus de datos.

♦ 2.10 — Tipos de memoria RAM

- **DRAM (Dynamic RAM):** necesita refrescarse cientos de veces por segundo. Más lenta y barata. RAM principal del sistema. Un bit = transistor + condensador.
- **SRAM (Static RAM):** no necesita refresco. Más rápida (10–30 ns) y cara. Se usa como caché. Un bit = flip-flop.

Ambos tipos son volátiles (pierden contenido al apagarse).

♦ 2.11 — Tipos de módulos y encapsulados

Nombre	Descripción
DIP	Chip individual con dos filas de pines. El más antiguo.
SIMM	Placa con chips RAM. 32 pines (3.5") o 72 pines (4.25", hasta 64 MB). Un lado de contactos.
DIMM	Similar a SIMM pero con 168 pines. Doble fila de contactos independientes.
VRAM	RAM de video: puede ser accedida simultáneamente por procesador gráfico y monitor.
RAM Disk	RAM configurada para simular un disco. Miles de veces más rápida, pero volátil.

♦ 2.12 — Evolución tecnológica de la DRAM

Tecnología	Característica principal
FPM (Fast Page Mode)	Diseño más común. Selecciona la fila una vez para acceder a múltiples columnas.
EDO (Extended Data Output)	Mejora FPM ~10%. Mantiene activo el buffer de salida hasta el próximo ciclo.
SDRAM	DRAM síncrona. ~20% más rápida que EDO. Entrelaza dos matrices internas.
DDR SDRAM	Double Data Rate: transfiere en flanco de subida y bajada del reloj → doble ancho de banda. Hasta 1 GB por módulo.
DDR2	4 bits por ciclo. Mayor ancho de banda que DDR, pero mayor latencia. DIMM de 240 pines.
DDR3	800–1600 MHz efectivos. Voltaje 1,5 V (DDR2: 1,8 V / DDR: 2,5 V). Consume ~40% menos. Físicamente incompatible con DDR2 (muesca en posición diferente).
PB SRAM	Pipeline Burst SRAM. 4–8 ns. Usada como caché L2.

Tendencia: ↑ velocidad, ↓ voltaje, ↓ consumo, ↓ calor generado.

UNIDAD 3 — Interrupciones en 8086

La Unidad 2 mostró al 8086/8088 ejecutando instrucciones de forma lineal, un programa detrás de otro, usando sus registros, la segmentación y el bus de direcciones de 20 bits; la Unidad 3 rompe esa linealidad: explica cómo ese mismo procesador reacciona ante eventos externos o internos —un dispositivo de E/S, una división por cero, un pedido explícito de software— reutilizando lo ya visto (el stack para salvar contexto, los registros de segmento CS:IP, la zona baja de memoria) pero agregando el mecanismo concreto que les da uso: la tabla de vectores y el ciclo de atención a interrupciones.

◆ 3.1 — Hardware de soporte del sistema

El 8086/8088 necesita circuitos extra para armar un sistema completo, porque los buses de datos y direcciones se multiplexan en los mismos pines, por lo que se requiere lógica externa para demultiplexarlos.

Señal RESET

- Activa en nivel bajo.

- Limpia registros y pone la dirección efectiva en FFFF0H (CS = FFFFH, IP = 0000H) en el 8086.
- Ahí se instala la ROM BIOS.
- Nota: a partir del 8086 y procesadores posteriores esta convención cambia a CS = F000H, IP = FFF0H (da el mismo resultado físico, FFFF0H, pero deja más espacio de código dentro del mismo segmento sin tener que recargar CS antes de pasar a modo protegido).

♦ 3.2 — Direccionamiento de E/S

Intel separa el espacio de E/S del espacio de memoria. Se usan señales IOR# e IOW#, e instrucciones distintas para cada acceso:

```
MOV AL, [BX] ; acceso a memoria
IN AL, 2Ch   ; acceso a E/S
```

E/S mapeada en memoria	Espacios separados (Intel)
No necesita instrucciones ni pines especiales	Set de instrucciones más chico
Todo el mapa de memoria disponible	Instrucciones de E/S más pequeñas y rápidas
—	Fácil distinguir accesos E/S en ASM
—	Menos hardware para decodificar E/S

♦ 3.3 — Tipos de interrupciones

Cada interrupción lleva un número que identifica el servicio a invocar.

- **Hardware:** generadas por dispositivos de E/S.
- **Internas (traps):** ej. división por cero (DIV).
- **Software:** instrucción INT (llamadas al sistema).
- **No enmascarables (NMI):** prioritarias, no se pueden deshabilitar (ej. Parity Error, Power Fail). Llegan por la línea NMI del procesador y no pueden desactivarse con CLI.

Las enmascarables se controlan con el flag IF:

- STI → IF = 1 (habilita interrupciones enmascarables)
- CLI → IF = 0 (deshabilita interrupciones enmascarables)

♦ 3.4 — Tabla de vectores de interrupción

Las direcciones 00000H a 003FFH están reservadas para la tabla de vectores de interrupción. Hay 256 tipos posibles. Cada entrada ocupa 4 bytes: primero los 16 bits de IP (offset) y luego los 16 bits de CS (segmento).

Cálculo de la dirección en el vector:

- Dirección del IP del vector $N \rightarrow N \times 4$
- Dirección del CS del vector $N \rightarrow N \times 4 + 2$

Vector	Descripción
0	Divide Error (dir. 0x00 / 0x02)
1	Single Step (dir. 0x04 / 0x06)
2	NMI (dir. 0x08 / 0x0A)
3	Breakpoint (dir. 0x0C / 0x0E)
4	Overflow (dir. 0x10 / 0x12)
8–31	Reservados
32–255	Disponibles para usuario

Aclaración: en la especificación de Intel el rango 8–31 está reservado, pero en la arquitectura PC concreta el controlador 8259 remapea las líneas IRQ0–IRQ7 justamente a los vectores 8–15 (por eso el TIMER usa el vector 8).

Las rutinas y punteros deben instalarse antes de habilitar las interrupciones.

♦ 3.5 — Controlador de interrupciones 8259

Chip intermediario entre los dispositivos y la CPU. Se programa con registros para: identificadores por IRQ, prioridades, y detección por flanco o nivel.

♦ 3.6 — Flujo completo de atención a una interrupción

1. Un dispositivo genera una señal de interrupción.
2. Si no hay una de mayor prioridad pendiente, el 8259 envía INTR al 8086.
3. Si $IF = 1$, el 8086 acepta y responde con INTA.
4. El 8259 coloca en el bus el identificador del dispositivo.

El microprocesador, automáticamente:

1. Salva las banderas en el stack (PUSHF).
2. Deshabilita interrupciones ($IF = 0$, $TF = 0$).

3. Salva CS:IP en el stack.
4. Salta a la rutina de atención (ISR), usando la dirección obtenida de la tabla de vectores.

El programador debe además salvar el contexto (registros que usa la ISR, con PUSH/POP) y tener cuidado con el stack. Al terminar la ISR se usa IRET, que restaura CS:IP y las banderas desde el stack — esto re-habilita las interrupciones si IF estaba en 1 antes.

♦ 3.7 — Instrucciones clave (concepto/lógica, no sintaxis de ejercicio)

Instrucción	Lógica	Descripción
INT n	PUSHF → IF=0 → TF=0 → CALL FAR tabla_int(n)	Genera interrupción software tipo n
IRET	RET FAR + POPF	Retorna de la ISR y restaura banderas
CLI	IF := 0	Deshabilita interrupciones enmascarables
STI	IF := 1	Habilita interrupciones enmascarables
PUSHF	stack ← FLAGS	Salva registro de banderas en el stack
IN AL/AX, op	AL/AX = I/O(puerto)	Lee de un puerto. Puerto fijo (0–255) o vía DX (0–65535)
OUT op, AL/AX	I/O(puerto) = AL/AX	Escribe a un puerto. Igual esquema que IN

Operadores del ensamblador para instalar vectores:

- `offset etiqueta` → devuelve el desplazamiento (IP) de una etiqueta.
- `seg etiqueta` → devuelve el segmento (CS) de una etiqueta.

♦ 3.8 — Acceso Directo a Memoria (DMA)

La E/S programada y por interrupciones requieren intervención activa de la CPU, limitando velocidad y "atando" la CPU a la E/S. El DMA resuelve esto con hardware dedicado (chip 8237).

Flujo:

1. La CPU programa el controlador DMA: tipo (lectura/escritura), dirección del dispositivo, dirección de inicio en memoria, cantidad de datos.
2. La CPU queda libre para hacer otra tarea.
3. El DMA realiza la transferencia.

4. Al finalizar, el DMA interrumpe a la CPU.

Robo de ciclos: por cada palabra transferida, el DMA toma el bus momentáneamente. La CPU queda suspendida solo si necesita el bus en ese instante — no es una interrupción completa, no hay cambio de contexto.

Señales clave del 8237:

Señal	Dirección	Significado
DREQ	Dispositivo → DMA	Dispositivo pide transferencia DMA
HRQ	DMA → CPU	DMA solicita el bus
HLDA	CPU → DMA	CPU cede el bus
DACK	DMA → Dispositivo	DMA confirma al dispositivo

♦ 3.9 — Comparación de técnicas de E/S

Técnica	CPU durante transferencia	Cambio de contexto
DMA	Libre, solo suspendida si necesita el bus	No (solo al finalizar el bloque)
E/S Programada (polling)	Ocupada esperando	No
E/S por Interrupciones	Libre, atiende al terminar cada dato	Sí

Identificación del manejador a ejecutar (complemento de 3.5):

- **Múltiples líneas de interrupción:** una línea física por dispositivo.
- **Múltiples líneas codificadas:** se codifica el identificador en varias líneas en lugar de usar una por dispositivo.
- **Por software:** el procesador, en su ciclo de interrupción, chequea el estado de cada controlador mediante un programa.
- **Por hardware:**
 - **Vector de direcciones** (caso 8086): el hardware entrega directamente la dirección del manejador.
 - **Vector de instrucciones** (caso SPARC): el hardware entrega una instrucción a ejecutar.